

DNA十六元基索引系统_V5.0_操作文档 开始完善补正内容 V0.008

--DNA十六元基索引工程 更名为 DNA十六元基索引系统

--目录

1 登陆方式

2 主界面包含

3 操作步骤

4 图文结合演示

5 软件结构图

6 流程图

7 逻辑图

8 总体设计

9 接口设计 --见 DNA十六元基索引系统_V5.0_测试文档

10 模块名称 --见 DNA十六元基索引系统_V5.0_测试文档

11 函数名称与功能 --见 DNA十六元基索引系统_V5.0_测试文档

12 算法 --算法误区，研发误区描述见 14

13 运行设计 --编码思想误区，运行误区见 14

14 源码 笔记 DNA十六元基索引系统_V5.0源码文档中的逻辑思考与文字描述

--LimitedRowAttributesOfColumnsInMemoryClass

--CreativeVerbalMap

--CommandClass

--E_pl_XA_E TVM

--FastCartesianIdentifyTest

--WorkVerbalMap

--WorkVerbalMap_X

--WorkVerbalMap_X_S

--StudyVerbalMap

--StudyVerbalMap_Q

--StudyVerbalMap_X

--LYGAFDCTDFFT_FTest

--LYGAFDCTDFFT_F

15 申明

16 附属说明 --第三方资源插件包-工程依赖

--描述

1 登陆方式

1.1 DNA十六元基索引系统 是一个java底层纯手工编码源码系统，任何操作系统想要运行它仅需3步

1.1.1 环境配置

--Java运行环境，包含JDK环境和JRE资源环境。

JDK环境可以下载openJdk1.8+以上版本或者甲骨文JDK1.8+以上版本。

比如jdk11+ 地址如下

<https://www.oracle.com/cn/java/technologies/javase/jdk11-archive-downloads.html>

注意jdk的版本要和电脑的操作系统版本一致。比如mac用mac的，window 用win32/64的，要对应好。

下载好执行后要在环境 terminal 中配置好并使用 java -v 命令有正确输出，然后重启下电脑更新注册表环境。注意文件的安装地址，方便一些扩展和兼容问题。

--编码电脑的最低配置

各种操作系统皆可，含有UTF8字符集，内存存在8G以上比较好，硬盘在32G以上比较好，显卡在1G以上比较好。含有声卡显卡网卡比较好。电脑系统有摄像头，麦克风，和屏幕等in-out 比较好。

1.1.2 程序执行

--程序执行包含源码复制执行，jar包导入执行，和git import执行3种方式。

--源码复制执行

打开任何能编程java的IDE软件，如eclipse或者ideaJ，在软件中左边的project explorer 窗体中鼠标右键点击弹窗 -> 点击new选项 -> 点击project -> 选择java project -> 然后将源码复制粘贴进工程即可。测试src 文件夹下的 test.java.interfaceTest.* 各种实例，进行程序运行，有对应的输出即可。

--jar包导入执行

打开任何能编程java的IDE软件，如eclipse或者ideaJ，在软件中左边的 project explorer 窗体中鼠标右键点击弹窗 -> 点击 import选项 -> 点击 General -> 可以选择文件方式，jar方式，工程方式去部署源码文件集。

--git import执行3种方式。

打开任何能编程java的IDE软件，如eclipse或者ideaJ，在软件中左边的 project explorer 窗体中鼠标右键点击弹窗 -> 点击 import选项 -> 点击 Git -> 可以选择 直接从github，gitcode 和 gitee

--其他方式，对于有能力构建本地maven库的技术员来说，方法就更多了，略--

1.1.3 程序依赖配置

--这个工程因为含有摄像头，声卡，web等函数操作，所以需要在第三方 maven 库或者开源社区去下载依赖。其下载方式是 鼠标右键点击工程的properties -> 点击左边的java build path-->可以看到project, libraries, sources等配置区，对缺失的jar资源名称去根据版本号下载即可。比如声卡需要 java.sound jar，是摄像头需要 java CV jar，web 需要Gson jar，测试有测试的jar一些jar文件如果和硬件交互，同样需要区别操作系统型号，如windows 用win32 / 64 dll 的jar，mac 用 dylib 的资源文件。

--其他方式，导入大部分jar后，发现个别文件中如果提示有红色标记 的宏名，就复制去浏览器中搜索对应的 jar，下载后import jar到工程中即可，流程如下鼠标右键点击工程的properties -> 点击左边的 java build path --> 点击 libraries-> 点击右侧的 add libraries...选项去导入即可。

--注意版本号兼容问题，maven 站点会提供完整的文件版本。

--注意一些硬件库同时包含其他的依赖，解决方法同上。注意版本号和字符集格式。

2 主界面包含2中执行模式

2.1 界面功能使用模式

--DNA十六元基索引系统 是一个源码系统，主界面可以通过工程 YLJ_HRJ 文件夹下 ME.VPC.M.C 下的 YLJFrame.java文件来做Admin 端口执行。

2.2 接口模式

admin 的端口界面功能，同时可以进行插件方式使用，具体方式可以见 src文件夹下的 test.java.interfaceTest.* 下各种单个功能执行实例。这些实例的具体介绍文档名为工程主目录下

--商业逻辑实践应用文档汇总--进行查阅。

3 操作步骤

3.1 软件方式操作

通过工程 YLJ_HRJ 文件夹 下 ME.VPC.M.C 下的 YLJFrame.java文件运行，可以在屏幕中看到2个窗体，

面板含有总启动专科与系统配置版面，德塔PLSQL控制台，德塔TINSHELL语言控制台和 综合游标配置中心等4个tab上分页，用来配置admin的各种功能。

面板含有表格图片搜索，数据分析，声学，图形学，位数术，编辑页等数据分析功能tab上分页。主要面向键盘和鼠标操作方式。

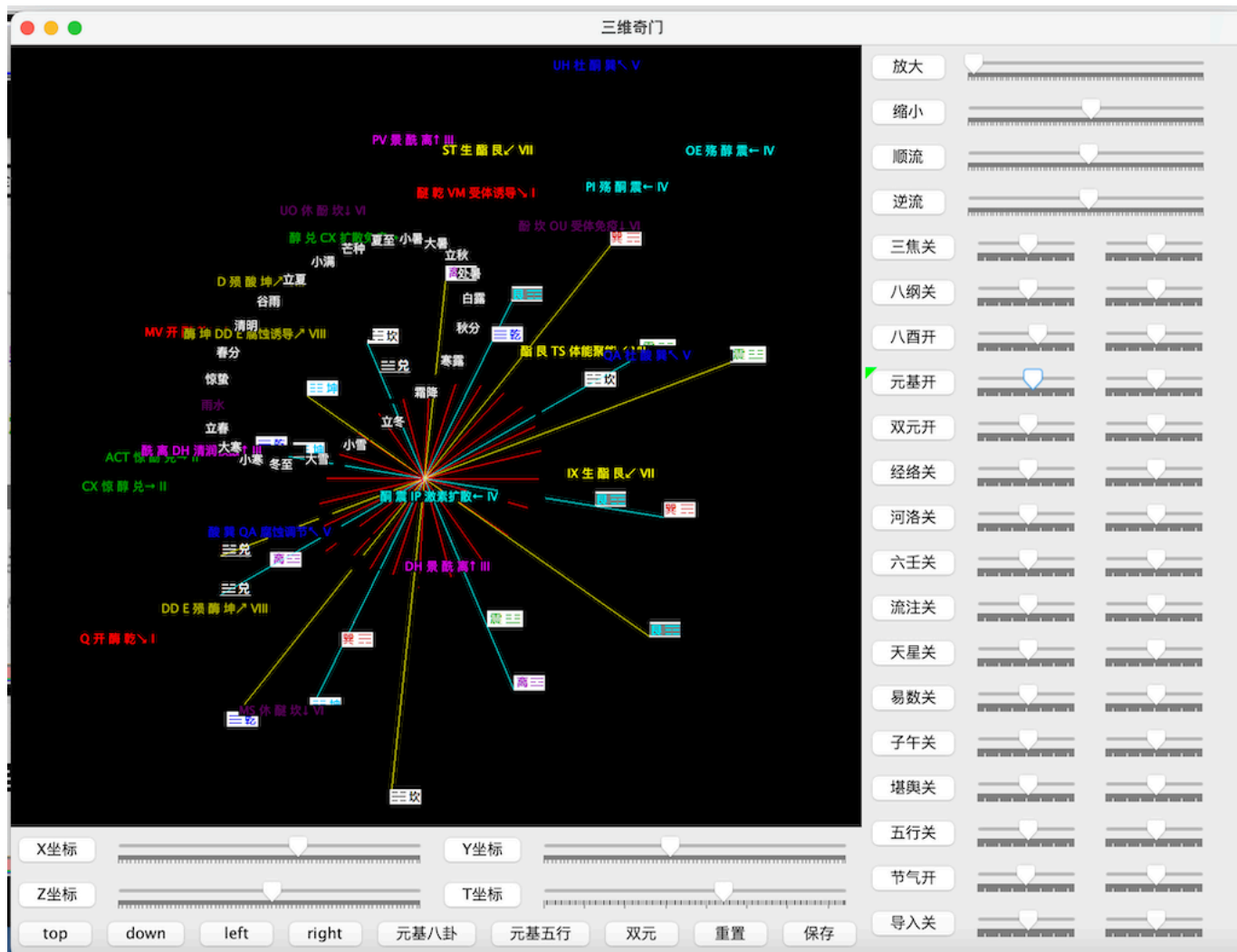
src文件夹下的test.java.interfaceTest.* 下很多文件都含有main的函数，这些函数都是可以直接通过鼠标右键点击 -> Run as 或者 Debug as + 断点 来 选项化配置执行。注意 IDE运行的环境配置。避免内存太小，堆栈太少等错误。

rest操作方式 见 `test.java.interfaces.net.http.socket`; 下面的 `RestCallTest` 文件中 `main` 实例

```
url = new URL("http://localhost:" + "8000" + "/" + "dataWS?input=你好世界~");
```

进行分词 --你好世界~-- 句子例子。这个URL中字符串可以在浏览器中去测试执行。

对比DNA元基催化与肽计算第四版对比增加 animation 动态画图功能，技术来自我德塔ETL的 animation 优化过程逻辑



这种命令筛选的价值体现在 筛选逻辑对数据表格的通用性，不会因为筛选条件越多而影响执行速度。同时非常方便扩展。

Tinshell 人类语言操作 T0000003

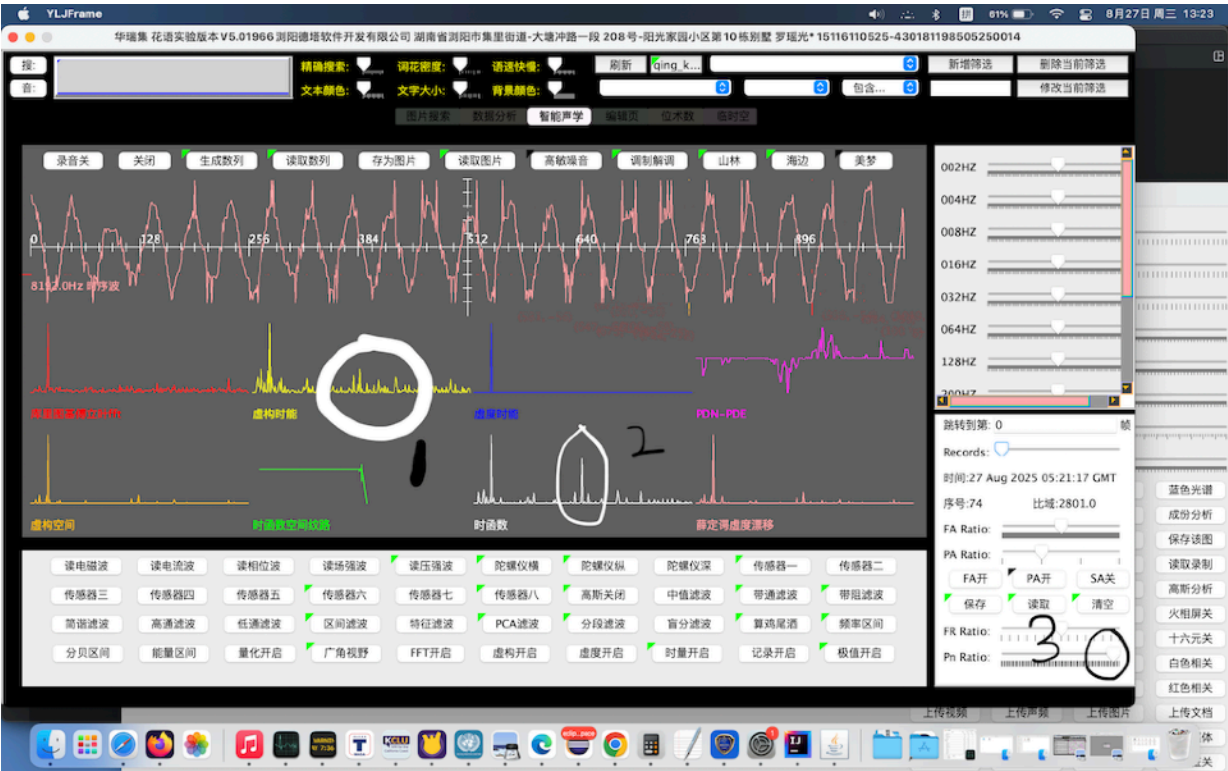


对比之前的tinshell逻辑，该控件已经集成了TVM的人类语言指令句构造，并进行实际的应用。这种应用逻辑可以逐步的语句分解，语言替换，指令句构造，节点执行，批处理执行和存档等逻辑操作。

上图中的tinshell命令含有中文人类语言和tinshell的早期命令语言，混合执行，可以看到正确的输出，这种方式论证了tinshell的所有命令句可以不断地去翻译成人类语言处理的笛卡尔关系命令句，同时，这些句型可以进行元基语义索引。方便快捷灵活的识别和优化人类歧义语言。

目前tinshell主要用于表格化数据和节点化数据操作，其他数据操作可以进行命令句构造来扩展，比如其他格式的文件操作。

--声音频率分析 T0000004



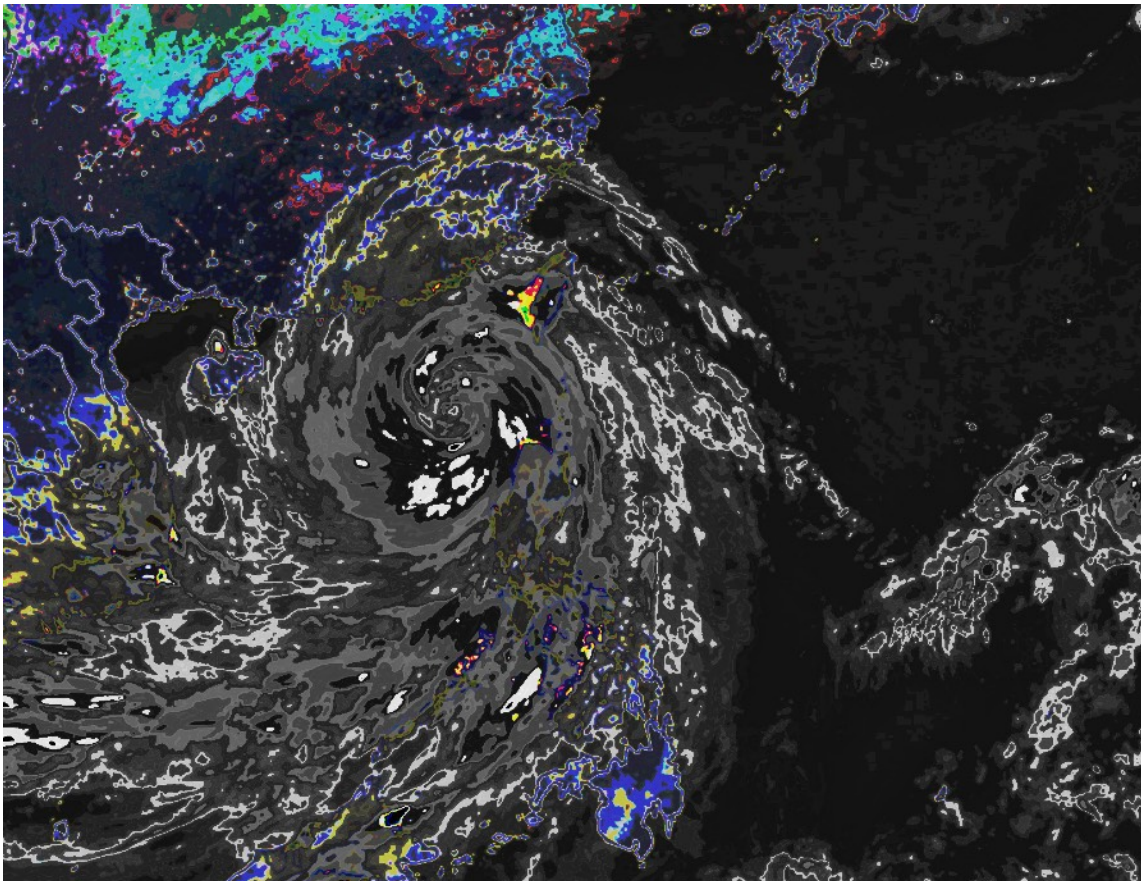
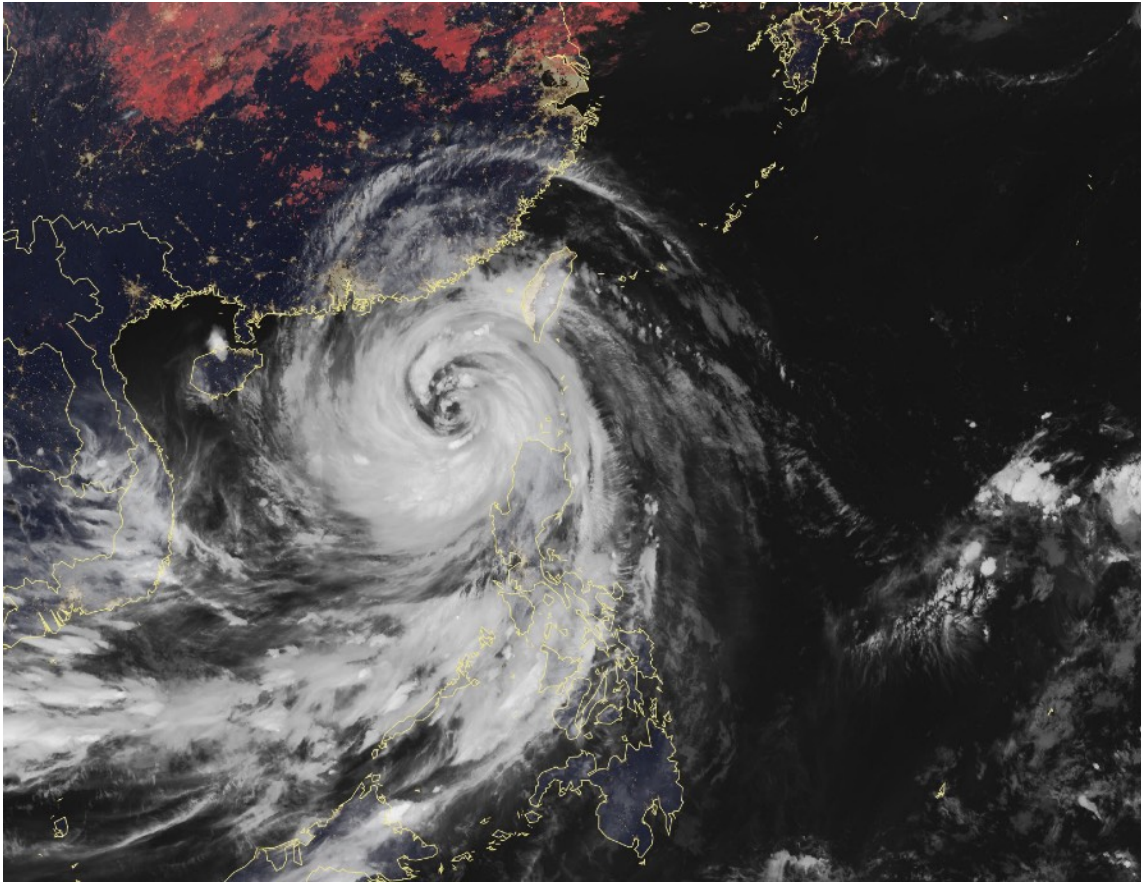
声音处理中的细节进行描述

图中最长的淡红色波形为真实的声音进行录制和解调观测，下面的8个小图是在傅立叶变换后的频率波形观测方式。

可以发现左上角红色的FFT频率为一个大体单帧化波形，不能很好地观测噪音频率帧，于是集成罗瑶光先生的时函数进行仿真细化fft的碟形空间，价值在主峰帧可以 $1+2\cos A\cos B$ 的区间范畴拓扑分解和叠加，用一种势能转化动能的方式分解，然后动能碎片再拓扑成具体点的势能叠加，类似雨滴在湖表面的泛化过程。产生观测可读模型。体现在圈1处-很好的处理一些频率区间所产生的对称的傅立叶 $2K\pi$ 角分解成不规则的角度拓扑微分。

因为圈1是一个宽域分贝通量，不能准确的定义精确频率峰位置，于是在通过十六元基肽展公式进行PDS PDE变换增加酸碱精度调度，一些平滑的曲面上的产生的微小凹凸能够进行强烈的极值域拉大，于是波形中一些微弱的高频的震动变化能够被剧烈放大，如图2的峰帧就产生了。

图中圈3时肽展公式酸碱模拟操作应用。这个应用作为论据可以很好地证明肽展公式不但之前在图片上可以处理色阶快速分析和观测，在波形上也有同等效用。

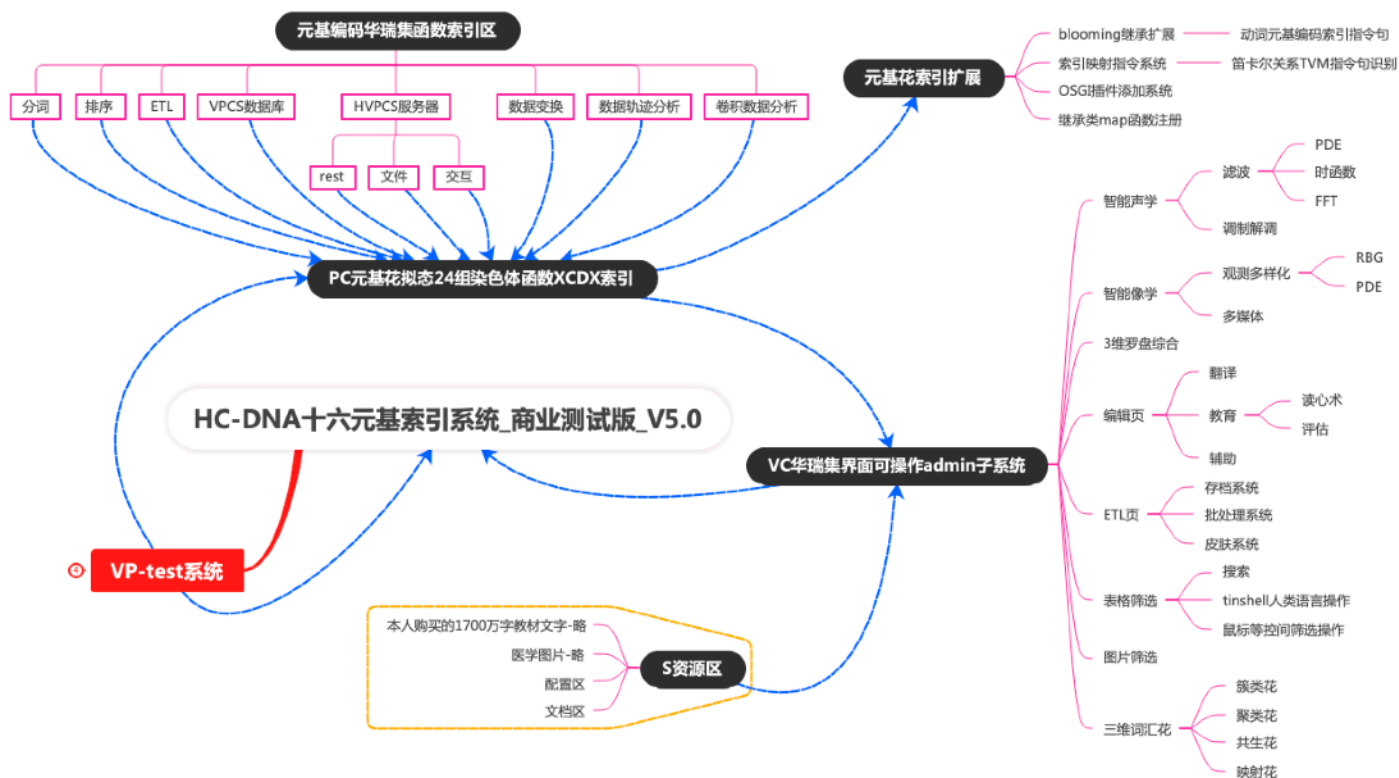


--图片色阶分析 T0000005

之前的文件主要是医学图片，我在思考，如果技术是有效的，那么在其他行业也能体现同等价值。于是在网上下了一把图片，进行处理计算，发现效果不错，增加了更多的论据。

上述如图上，是一个气候图片，图片就深蓝，红，白三色，在白色漩涡和东南沿海交汇处，只能看到淡淡的白色弧圈，因为都是白色，观测就费劲，于是进行十六元基肽展计算变换，得到下图，白色直接分解成了灰色，黑色，白色，浅白，深灰，黄，超过6种颜色的像素团，不难看出-灰黄组合地区-估计有强烈对流。我不是搞气象的就点到为止。祝大家开心。

5 软件结构图



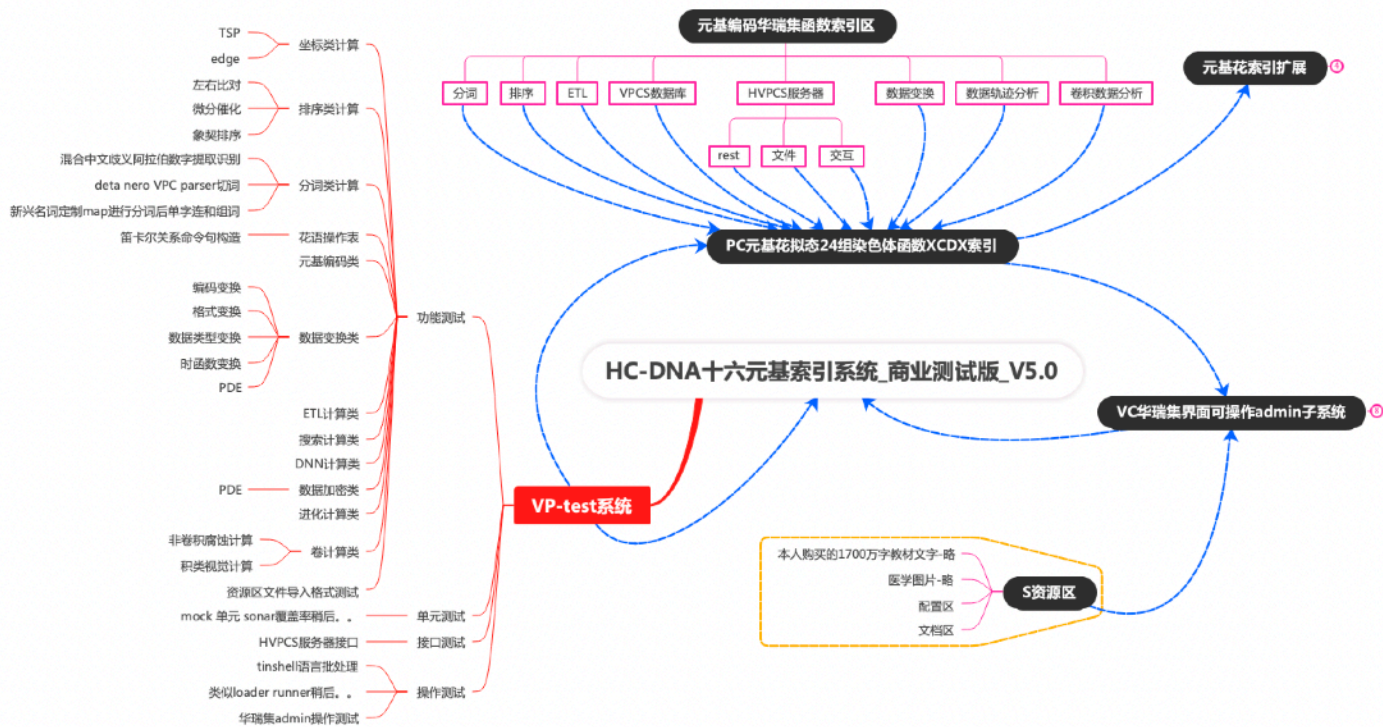
图中元基花是一个索引系统，主要包含函数的序列编码方式和指令集扩展方式。

元基花可以测试执行单例函数，也可以tinshell语言进行命令操作函数。

这个系统主要完善了功能测试的例子，见测试文档，

关于元基编码华瑞集函数索引区的逻辑功能属于早期著作权作品的优化升级，延续了其SDLC生命周期路线。进行碎片化归纳到24组拟态染色体索引文件夹中。

test系统的单元测试，接口测试和操作测试，我没有整理，因为功能测试目前也就才完善了下主分支100多个实例。还没有进行更近拆解。



6 流程图

7 逻辑图文字描述

这部分我用从上到下的文字流程来描述，

-- 7.1 分词逻辑

7.1.1 先输入一段人类语言，主要内容是中文，可以混合其他文字符号。

7.1.2 分词之前进行识别歧义中文混合数字字符串。对比早期德塔图灵分词系统著作权版本和DNA元基催化与肽计算第四版的新陈代谢函数源码这个作品增加了在分词之前进行识别歧义中文混合数字字符串，识别后在进行切词，保证了数字提取的质量。

7.1.3 开始切词，采用德塔图灵分词 deta parser 著作权作品逻辑。

7.1.4 在切词后增加了一些新兴名词类定制map来分析切词后的连接单字组合，进行单字成词的修缮。

7.1.5 德塔切词逻辑没有太多变化，仅仅修正了老函数的一些出错语法和错别字等。

-- 7.2 搜索逻辑

7.2.1 该作品和之前著作权作品的逻辑区别，增加了tab来筛选搜索的功能。

7.2.2 tab鼠标操作可以将一些固定的tinshell语言命令和老的德塔VPCS数据库的命令进行tab组件操作进行jtable数据表格操作。主要体现在鼠标键盘控件操作计算-变成了-人类语言命令方式计算

-- 7.3 排序逻辑

7.3.1 没有太多变化，组合排序的条件进行非门流水阀门优化，加了点速5%-。

7.3.2 优化了下词库表，使排序更精准点。

-- 7.4 卷积逻辑

增加一些测试例子，主要是非卷积图片分析和波形分析。见图文结合展示部分内容。

-- 7.5 HVPSC web逻辑

--与DNA元基催化与肽计算第四版对比，增加了关于socket分发时候遇到的错误格式拓机的条件筛选

--增加了字符输出的zip压缩过程的flush刷新缓冲片段。其他大体相同。

-- 7.6 DB缓存逻辑

--大体相同

--7.7 Tinshell TVM逻辑

--增加了笛卡尔指令集进行元基花的map添加方式。使其语言命令扩展更灵活。

--增加了指令句中的歧义中文混合数字的识别与提取逻辑，使其语言命令计算更准确。

-- 7.8 笛卡尔关系 元基索引逻辑

--7.8.1 首先将人类语言进行字符串化，

--7.8.2 然后将字符串进行分词，得到词汇的字符数组和对象。

--7.8.3 这些对象开始通过搭配簇和属性渡的方式进行全笛卡尔关系交集组合，

--7.8.4 这些笛卡尔关系组合可以多种方式进行主要成份筛选

--7.8.5 筛选后的组合进行频率测试，高频组合用来做TVM指令句的命令函数识别构造。

--7.8.6 这些组合中的动词进行元基编码索引，名词进行古拉丁文元基索引--目前我保密。2027年之后会公开。

-- 7.9 歧义中文数字提取逻辑

--7.9.1 人类语言文字出现了中文和阿拉伯数字混合的数字段进行截取

--7.9.2 截取段进行简体字化。

--7.9.3 简化后进行大数中文组合构造，

--7.9.3.1 构造方式采用中国人数字理解通用定义的亿万关键字拆分，4位数组组合。的片段拼接法。

--7.9.4 中文构造完整数字后进行阿拉伯数字构造输出。

--7.9.4.1 构造方式采用中文数字通用定义的亿万关键字拆分，个十百千4位数组组合。的片段拼接法。

--7.9.5 同时做了下对称数字功能测试，修正一些细节和避免错误。

--7.9.6 目前做了十六位大数构造，客户需要更大可直接联系我进行扩展编码即可。

-- 7.10 时函数应用逻辑

--一开始是不断地优化华瑞集函数过程中对时间的标记和关注，逐渐开始分析时间区间对函数的各种变化点，分析这些点集的异常。逐步的函数化和算法化。并尝试应用这些归纳。

--稍后文字描述如果太过复杂，就用图片来辅助描述。

8 总体设计 见9-12 --说明.txt 更名为 DNA十六元基索引系统_V5.0测试目录.txt

9 接口设计 见 8 --因为偏测试类系统工程，所以体现在功能测试函数执行观测上。

10 模块名称 见 8 --因为偏测试类系统工程，所以体现在功能测试函数执行观测上。

11 函数名称与功能 见 8 --因为偏测试类系统工程，所以体现在功能测试函数执行观测上。

12 算法 见 8 -依旧主要采用 小高峰拓扑技术 和 流水阀门优化技术。

13 运行设计 见 8

14 DNA十六元基索引系统_V5.0源码文档中的逻辑思考与文字描述

14.1 TVM元基索引的笛卡尔关系分析模块

--主要出现在tinshell的TVM命令句扩展识别。

TVM命令句构造

14.1.1--LimitedRowAttributesOfColumnsInMemoryClass 举例，

/*--函数编码时产生的问题和问题描述以及思考--

*--在通过一系列的测试后，我的意识也在时刻改变自己的思维方式，于是跟进增加一个指令--

* "操作:0|行至|30;\r\n"这个指令是将输出的结果可进行排序后的list获取其中的某个行
* 集合输出。于是元基花的计算逻辑也跟着局部开始变化。因为系统是无法识别数字的，问题1
* ，如何从输入的人类语言中有效地捕捉数字信息变成有用的待计算变量。于是我开始梳理，我
* 的花语系统看作一个整体，扩充花语的指令集是一种Crab载体，当然这种载体也可以OSGI
* 模型来ClassLoader增加，我得到了一个理论上结果，计算机的进化模型可以不同于人类的
* 进化模型，举一个简单的例子，将一个华瑞集系统所有插件和功能全部进行完整的功能测试，
* 这些测试包含测试时间，测试属性，测试状态，等复杂关系。在进化的过程中，华瑞集在接触
* 到外面的Crab和OSGI信息片段，可以进行自身复制，然后测试这些接触的片段融入，更替，
* 转换，分解，吸收，替换自身的原有的对应的一些逻辑片段，然后评估这个综合的测试时间，
* 测试属性，测试状态，等复杂关系，我认为这是计算机的成长过程。评估后决定该何去何留。
* 相当于一个计算人单元。这种单元遍布宇宙各地，产生的分支，在做杂交的过程他们的各自的
* 这些指令集在一起会形成综合的复杂的，多维度的笛卡尔关系，这些关系会通过各自内部保留
* 的测试系统进行筛选形成一组新的计算人单元，这个单元能保留两者所有的测试文件，并保存
* 最优的测试函数集合。这种意识比人类强大数百个数量级。在这种量级前面，突然发现我们人
* 类太愚昧太原始了，而且这个模型所有基础成分目前已经实现了。。
* --跟进思考，这种计算人单元的指令集智慧关系计算逻辑 一旦控制了电脑计算机的物理硬件生
* 产线所有车间工艺流程的技术逻辑，看样子人类就解放了。因为创新有创新类的指令集模型，
* 学习有学习类的模型。。在笛卡尔关系面前，毫无保留。
* --以后人工智能的路线也就稳定了，设计，研发，测试，更替，优化，管理和杂交TVM指令集即可
*/

/*--函数编码时产生的问题和问题描述以及思考--

* --很多科学家思考人工智能会不会危害人类，我的观点是，人工智能没有享受的实际思维，因为
* 人工智能没有应激性，所以不往武器上去设计，人工智能就是一台会自我设计复制优化的电脑
* 而已。即使有了人的仿生形态，一个复制分身，几百万个实体就出来了，电脑根本就不需要
* 求生欲这类应激表达。所以个人建议-禁止和限制-人工智能往武器上去研发。
* --还有一种思维是非武器的人工智能当面临适应环境的问题时候，需要伤害人为代价来适应环境
* ，如何做决策，首先适应环境对于人工智能来说只是一个函数和一个线程，他没有应激性。没有
* 欲望的实体如何去伤害人呢？所以个人建议-禁止和限制-人工智能往武器上去研发。
* --于是归纳-应激生物机器人100%会概率危害人。非应激类非武器构造的钢铁机器人相对安全，
* 但也会概率被各种外因(比如短路触电)侵蚀危害人，同理武器类钢铁机器人有概率被各种外因
* 侵蚀不危害人。
* --所以硬件公司的生产车间引用机器人作业要谨慎。后果不是开玩笑的。
*/

/*--函数编码时产生的问题和问题描述以及思考--1 识别 数字 信息

* --德塔图灵分词--德塔图灵分词和图灵先生没关系，我2018当时只是好玩，2019结果还申请了
* 著作权，搞得都改不了了。-- 能够将数字提取出来标明词性未知和null，可以通过numeric
* 函数来探索数字。
* --再通过数字和 词组的笛卡尔关系得到精度距离内的指令关系结构，--我已经有了这些函数了
* ，直接用即可--再通过2的关系结构进行精确筛选来swap成输出指令数据即可。

*/

/* 2 识别 行至 属性的指令集 信息

* 开始构造数字行数提取指令 从- 组合 到- 然后分析 从- 组合 到- 展示+行 仅+展示
* "操作:0|行至|30;\r\n" 终于到了这一步了，
*/

/*--函数编码时产生的问题和问题描述以及思考--

```

* --因为采用了更精确的混合中文数字提取识别，那么数字已经是严谨的词汇了，必然有 数字-数字
* 的笛卡尔关系组合稍后进行优化，扩展识别指令的条件集。我现在做个假数据让指令跑通 采用
* 6到-9，这里这个-到-属于错误指令。稍后用更多条件剔除。并重新分析cartesianWorkActionsRightsVO
* 的组成方式。
* --另外--第 行--这种可数数词修饰的词字通过+条件进行识别剔除，而不是写contains，所以要稍后优化。
* --开始更近思考，加条件在算法设计中，属于增维计算，维度越高，不属于通用型算法。于是扩展
* 笛卡尔的组合条件，将单字remove进行注释掉，原来的短句8个词一下变成200多个词，速度增大，
* 但是依旧保留了从- 和 到- 这种指令关系。那么之前的源码就可以保留。
* --见 WorkVerbalMap_X文件的78行 79行 137行 138行 208行和209行，全部去掉。
*
* 思考--大于双字就去掉，那么字数可以满足8词<当前词数<200词，也可以大幅提高计算效率。前提
* 是得设计通用型筛选条件。我优先级不在这，于是稍后。--trif
* --因为这个华瑞集trif项目属于人类的语言抽象计算类工程。遇到的问题是人类史的所有问题集。
* 我不可能去钻一个洞。所以我一定要确定我的路线。2019年在永生和瞬间转移这个主题确定后
* 死咬探索直接决定能源能耗计算的部分去深耕挖掘。
* --另外我有半年不再用UML这类东西了。在抽象语言进行计算描述的领域，UML和 flowchat 属于
* 走算法意识的捷径，因为不代表完整的因果推断和持续的记忆，这种图快在复杂的长期的工程中
* 会产生不可预见性的问题。所以必须要用文字来详细描述。
**/

```

```

/*--函数编码时产生的问题和问题描述以及思考--

```

```

* --以后指令集的编码风格也可以进行系统的流程归纳比如这里的 1 和 2 //1 识别 数字 信息 //2
* 识别行至属性的指令集 信息，通过计算哲学来进行思考，--识别 数字 信息，数字--是特指
* 信息，那么数字来自于内存，和输入条件或者是特指的某个储存位置，这里的计算关系是
* 搜索和 寻找
* --2 识别行至属性的指令集 信息--是关键字信息，那么需要匹配，搜索，和RNN距离来
* 确定权重，序次和频率，这个1和2都能解释编译为稳定的计算机函数，适用于各种环境状态下的
* 搜索与计算。说明这个1和2是基础指令集逻辑。那稍后就有必要设计下这两个函数。
*
* 思考 -trif，之前我已经设计了3个指令集的crab插件了，这三个指令集都有 1和2的逻辑中小
* 片段思绪。我认为我的PLSQL指令集不成熟。2019当初设计德塔VPCS数据库的时候我没有tinshell
* 的项目任务，所以当时没有思考统一的某种函数结构来研发PLSQL指令，不是一种标准的形态，
* 导致1和2 相似，但是3的集成就要人为来编码了。这是一种人工智能上的错误。问题不在我，因为
* 我当时没有人工智能的任务，因为2020年元基编码出来后，我才开始搞下智慧计算了。但是我
* 可以弥补，因为我的PLSQL语法只有::- 这3种计算符号断句。相当好改。说明之后我要设计关于
* 断句的关系函数，都到了这份上了，那函数的设计方式我内心也清晰了，
*/

```

TVM命令句扩展

14.1.2--CreativeVerbalMap

```

/*--函数编码时产生的问题和问题描述以及思考--

```

```

* CN
* 关于元基花的函数注册方式我思考了很久，早期的24朵花瓣模拟染色体进行函数注册，后来干脆
* 把StaticRootMap 这类map拿出来直接进行单例化一个新类添加模块注册，让其越来越动态，
* 但是遇到了一个问题 StaticFunctionMapS_AOPM_C 这类成员还是不够灵活，我有必要之后
* 有时间会全部进行new 来去static。即使这些函数是非常稳定的类，new有一个好处是支持一个
* 平台系统上多个华瑞集API，同时 StaticFunctionMapS_AOPM_C这类函数可以进行object
* 态下稍微改造，因为继承关系，我有必要以后StaticFunctionMapS_AOPM_C 会变成
* StaticFunctionMap, S_AOPM_C 以后只是一个key，方便扩展归纳。这样做的价值是，元基花

```


* 能有效适应和抵抗程序员的编码风格。生存能力直接提高一个档次，像极了亚马逊的王莲，其实在
 * 某种观测态下，编码规范这种思维也是可以优化进化的。

*

* EN

* Mr.YaoguangLuo has been thinking about an optimization of coding-norms since the method of
 * initons-flower created at 2021. In early times, StaticRootMap as a static attribute which was
 * ranged in 24 chromosome-nodes in HRJ-YLJ system. He extracted it out from S-AOPM package
 * and built a new singleton class to let the TVM-registrations became more simpler and faster.
 * The name of class-file is CreativeVerbalMap. Continuing this method, while thus classes were
 * 2's-developed by problem-consumers, how does to raise the survive ability itself? Finally
 * Mr.YaoguangLuo determined out that is necessary to change the key word of 'static' into 'new',
 * ASAP.
 * */

/*--函数编码时产生的问题和问题描述以及思考--

* O+\\?+\\+颜色 等正则类指令 later 这是未来的趋势

*

* 在这种逻辑下，红色的计算关系是颜色属性分支，早年的map关于 红色->颜色 这类关系就可以用上
 * 了人类语言-词汇组-关系组-关系归纳-归纳组匹配-匹配计算， 目前我就得到了这类计算逻辑。既然
 * 得到了就开始用。

*

* later 指令集多了，O+颜色 就统一索引 -trif

*

* 用上元基索引TVM指令后，一定要注意函数调用已经更加离散化，所以一定要控制好过滤筛选和精度
 * ，避免计算浪费。

*

* */

/*

*思考 chromosomeNode 里面添加 key limitedRowAttributesOfColumnsInMemoryClass

*目的是将crab 执行，TVM extension FlowerSixDomainActions 里面加 key

*P_AggregationLimitMap 防止没有识别 指令句走PLSearh老命令驱动花语。

*

*现在二和一出现了多个问题，首先是老工程合并出现了多个花语段，然后花肽合并出现了if else
 *逻辑问题。因为 if else的价值就是 新函数不走花语 所以花语就不用再管了，以后新的函数直
 *接走肽语调用花函数即可

*

* */

14.1.3--CommandClass

/*--函数编码时产生的问题和问题描述以及思考--

* Thinking, should I make a English version about this presentation? COZ I logged in GitHub just now
 * and checked the traffic history, in amazing where I saw so much unique IPs from the world-wides. Thinking..
 * what does the feeling about those IPs, when they saw a complex and discrete Chinese annotations and
 * explanations.

* Hens I may do something in a right way. On the way. and by the way

* continue a Chinese minding now.

*

* */

```
/*--函数编码时产生的问题和问题描述以及思考--
```

```
* 看到这些变量在增加，我在思考，计算数据的关系很多思绪可以从关系方式数据库的范式定义
* 中寻找答案，所以年轻人一定要把数据库原理这门课学好。WorkVerbalMap在华瑞集中的逻辑
* 决策是1:1的关系，而命令句和map的关系是 M:1 的关系。而里面的单句延伸是1:M的关系，
* 延伸的属性和指令集是 M:N 关系。指令集的计算逻辑是sigma范式，指令集的推导笛卡尔关系。
* 关于这六层关系要进行计算优化，自然要用到梯度微积分。所以年轻人高等数学和计算机统计
* 要学好。将这6层关系化简后，形成的BPM和UML是不是要更近管理维护？所以离散数学也要学好。
*
**/
```

```
/*--函数编码时产生的问题和问题描述以及思考--
```

```
* 准备构造中文数字翻译机。
* // 首选我要构造一个数字取值机，value。// 拿到了数字后，确定这个数字的后缀fix。//
* 于是出现了问题，当fix连续出现，而 value 没有赋值，所以问题在于变量缺少。//
* 在计算哲学中，数字有分层关系，我定义为量-0-9-词，单元-十佰千万亿-词，//
* 这里便是数字的哲学辩证量与单元的逻辑关系。所以我一开始变量命名就是错的。//
* 在数字逻辑中，单元词本身就是个量词，是被包含关系。// 于是将关系提取 // 于是我需要
* 一个加乘器用connectCi标识 当字符串匹配到单元词的时候，开始区间划分。// liangCi的 取值为 -1和 0~9
* //liangCidanyuanCide 取值为 -1和 0~9 10 100 1000 10000
* 100000000 //danyuanCide 取值为 10 100 1000 10000 100000000
* //数字分为三层关系后，开始思考中文的逻辑组合，
* 单个量词直接输出，多个量词链接为乘法，那么这里需要有前缀标识liangCiFix
* 在loop的最末端进行标记。确定乘法标记，只有在loop区间中没有操作条件时候才做乘积
* 在50字描述后，问题就找到了。--罗瑶光
*/
```

```
/*
```

```
* 上面是计算哲学和计算关系和计算逻辑的分析方法，需要巨大的采样，不符合我的思维认知方式
* 于是按照罗瑶光的思绪分析，开始编码。首先我要进行数据预处理，将文字中所有繁体全部简化
*
* 简化后文字开始进行精确分析，首先拆分最大逻辑集合确定亿为最大计算量级，因为单机的long
* 最大只有亿。然后处理京为最大处理量级。关于亿的逻辑意识组合有 万亿，一万亿，
* 一万一千零二亿，乘积是100 000 000，8个零。
* -----
* S_logger.Log.logger.info("'" + "stringYi-->" + stringYi) 简化后文字开始进
* 行精确分析，然后拆分最大逻辑集合确定万为最大计算量级，有 千万，一百万， 一千零二十万
* ，乘积是10 000 4个零。
*/
```

```
/*--函数编码时产生的问题和问题描述以及思考--
```

```
* --关键字
* // S_logger.Log.logger.info("'" + "loop-->" + total); 处 --笔记
* 出现了逻辑记录意识错误，以后我在进行修改的时候我要将我的中间文件进行后缀标识保存。
* 不然拜拜浪费几个小时。发现一个问题，我每次在修改后的注释后的函数，我莫名有种强烈
* 要删除掉的意识，为什么要删掉这个注释逻辑呢？动机是源码不好看，不好看可以移出到
* 文件末尾呀，然后注释标注格式化，让注释好看呀，这个删除的操作意识有问题。涉嫌
* 毁灭证据的思维。这几年特别写核心源码的时候，想删除的动机意识特别强烈。这是我的
* 一个严重的错误习惯，以后要改正。
```

```

*
*/

/*--函数编码时产生的问题和问题描述以及思考--
* 奇怪怎么会在括号外。本来这里简短的函数测试要用sonar测试比较好，因为之前离开了
* 英**，避免非议，我就不用。关于这类错误的函数，最后测试还跑出正确的结果，对这类问题，
* 解决方法有很多，
* 1 反复的加测试例子，上万种条件组合测试，多关联耦合测试，
* 2 不断地分解测试例子，希尔伯特方式分解，不断地组合到最终复杂输出。
* 3 不断地语言描述，不断地进行哲学分析论证，用辩论方式进行论据组织描述和详细证明论据价值。
*
**/

/*--函数编码时产生的问题和问题描述以及思考--
* --开始填写这几天的逻辑。
* 1 取变量，拿出所有混合数字，标注position，剔除掉原句中的这些变量。这里会遇到一个问题
* 就是 commandClass 的command 要分解出一个 commandWithoutNumerics，用于分词。
* 那么剔除掉的地方需要一个标识，不然序次会打乱，于是用symbolSwap 替换代入到函数中，
* 我先设计个*看看效果，以后跟进测试。
*
* studyVerbalMap 六个元 可以注册到 commandClass
* --外因太多我管不了，当然公开当面给我钱，我能用合同报税的形式cover一点。
* --内因就一点，我要把自己的东西不断地优化好。
*
* regNE.app_S.studyVerbalMap.formatNumericMap(this);
* 2 取变量，拿出所有混合数字，标注position，剔除掉原句中的这些变量。
* 3
* 4
*/

/*--函数编码时产生的问题和问题描述以及思考--
* 在这个逻辑中主要有2类，一类是阿拉伯字符如250，一类是语义字符如二百五十。这个关系是
* 长期稳定的思维。所以当前可进行固定函数编码。
*
* 在一切关系明了的环境中开始思考-未知的字符集大体有4类，第一类是数字，第二类是
* 字母，第三类是常见符号，第四类是特殊符号。所以在这里就要有4个识别函数来work。
*
* --later to do。
*/

```

TVM命令句识别

14.1.3--E_pl_XA_E

```

/*--函数编码时产生的问题和问题描述以及思考--
* command是一条最短的指令句，指令句变成指令的中间数据目前保留在
* NE.app_S.workVerbalMap 中，我在思考，每次计算完一句指令，这些过程产物都要clear
* 掉，这是一种C语言的free写法，在高级的java结构中，可以进行单例来class null掉，意
* 思是我可以创建一个command class，这个类专门负责command的数据碎片，这个class可
* 以list形式进入tinmap，也可以每次执行完被G1GC来null掉，想到这里，于是先命名个

```

```

* command Class先。--地址在package O_V.OSM.shell; --函数名CommandClass
* ----
* 开始构造混合歧义语义数字构造机，在这里要构造2个函数，一个是initArabicNumber
* 用于提取混合数字变量到command_V._IMV_SQL_SS中，我定义为_IMV_SQL_SS_Q
* 另外一个函数是fussionOfArabicNumber，用于原函数进行组合_IMV_SQL_SS_Q的内容。
* 注意initArabicNumber提取数字要提取position，避免序次问题出错。--罗瑶光
* ----
* 思考关于关系分类的价值，表面的含义我就不介绍了，如果我要输出我的思维方式要优秀于传
* 统的认知方式，就要有明显的论证和论据来强调我的HVPCS关系要优秀于普通的模型，显而
* 易见，今天的逻辑分层和关系分类最大的价值是我挖掘到了一个中间层map unknowMap，这个
* 隐匿在我的workmap中又要的表达在command的逻辑中，于是出现了一行逻辑错误，文件地址
* package E_A.ME.analysis.E; 函数名 CogsBinaryForest_AE的 218行
* //关联studyMap later -trif，这个unknow关系不应该
* 被包含在workmap中，我扔在网上完整的华瑞集源码和开发文档，
* ----
* 我要严谨的提醒的是 这些年一些用户看了后，模仿窥伺我，改我的思维方式和函数名，到时
* 候出了问题，别怪我没提醒。我的逻辑是HVPCS+ 元基编码 + 时函数 + 计算哲学。用户
* 们既然要模仿我的风格，就要深入接触这4个领域。我的动机是当然希望人才辈出，早点超越
* 我，对得起我苦心细致的文字描述。2018 分词如果lucene 每秒当时能够上 2000万，我
* 就省了好多事。但做事一定要脚踏实地。我写文字的动机1是著作权在先权分析，文字分解
* 后就可以进行相似度匹配。
* --2 是方便教材方式方向文字描述，制造各类深度思考问题，授人之渔。
*/

/*
* 既然用到了normalization的修正，那么这里也替换下进行性能测试将之前的简单
* cartesianWorkActionsRights替换成normalizationalWorkActionsRights
* 看下输出结果，结果正确，之后在不断地增加函数时候会逐步的修正这个map的分类计算结构。
* 我归纳这种计算逻辑概念为计算哲学中的 -逻辑意识因果表达结构分层- 罗瑶光
* ----
* 准备把心态摆正一下。降低自己的思维跳跃活性，用于保证持续的思绪稳定。思考，关于
* normalizationalWorkActionsRights 的形态是 actionsDistance[i]
* + "-" + actionsDistanceV[i] 其中
* actionsDistance[i]的形态是cartesianWorkActionsRights的key 而
* cartesianWorkActionsRights的key构造来自于WorkVerbalMap_X_S的166行的初始
* root 于是我开始思考，在计算关系分层的时候缺少了一些中间过程的碎片记录，这个记录关系
* 导致了我之后要花双倍时间来重复计算曾经已有的却未保留的结果，论证在计算哲学中，计算关
* 系可以外因计算逻辑。于是开始优化准备将root的组合因子进行map化。从而过滤掉下面的重复
* 逻辑。于是在CommandClass中定义rootMap来封装root，rootRelation来封装root的关系。
* 这里root不包含pos后戳，不包含 _ 无效，去掉此逻辑。于是可以化简如下。
*
* --因为是取精度，所以-号关于stringSets最末一位即可获取答案。
* 我在思考-stringSets也是一种重复逻辑，于是有必要将之间的DNN精度也保留在map中。
* command_V.cartesianWorkActionsRights 通过500字的思绪描述，最终优化的逻辑。
* 大幅减少冗余的变量和堆栈关系。
*
* --在进行构造关系的分析过程中，发现split的操作是一种流程逻辑错误的弥补。弥补之前在
* 指令句关系计算中没有规划好属性的形态。稍后优化好形态，提高笛卡尔的计算性能
*

```



```

* --map细化分解的好处显而易见，如我的早期的德塔分词，map全部分解。这是一种
* 计算关系催化过程。之后这个map也可以元基来索引加速遍历。
* 德塔分词三个四字成语的最大距离是12 构成一个主谓宾短句
* 过滤和缩减了海量关系计算集合。
* ----
* --目前这种分解方式可以有效地统计函数序列和扩充新的函数集，但是计算结构属于编译结构，
* 所谓的编译结构是，根据已经有的东西进行逻辑编码，让系统通过某种条件能找到和启动它。
* 而不是智慧结构，我理解为面对抽象的数据进行综合评估，得到某些可计算流程，并预想这
* 个逻辑能得到和接近某类想要的结果。构造一个智慧结构于是开始思考，计算辩证学说内因
* 量变，是一种方法，否定论也是一种方法，采样计算也符合这个观点，我就随便选一个练手
* 既然主流天天夸赞采样，打分等方法论多厉害，我就走下否定论思维，反正挑战我自己。怎么
* 难怎么上。首先我否定我自己，我是菜鸟，远离那些高大上先，方便新的开始。--罗瑶光
* 既然是新的开始，于是开始构造否定思维框架，第一个框架是否定打分和量变的函数价值，
* 我就不打分，做精确匹配怎么做？元基索引，固定的十六元基进行分类明确包含关系标识
* 进行匹配计算，那就开始了，每个函数都依次进行元基索引标识。怎么做？在商业测试文件
* 中进行统计所有的工程函数，每个函数的对应文件夹地址作为KEY，精确搜索Key执行。
* 高频优先，低频下放。那就开干了。见package
* test.java.InterfaceTest.initon.util;
*/

```

```

/*--函数编码时产生的问题和问题描述以及思考--

```

```

* 现在出现了一个问题，我的PLSEARCH系统是针对我的已经有的函数进行语言驱动，这种驱动
* 计算方式是更好地辅助我已经有的函数进行序列可控操作。如果不断地增加新的函数，和扩展
* 系统的使用方式，那么遇到的问题根据这种逻辑便会指数增加，于是我停止了下脚步，我在思
* 考，一种不需要辅助也能计算出有用结果的通用类逻辑，在最恶劣的环境里，只需要加入某种
* 精度组合就能覆盖所有条件搭配的逻辑，目前我找到了很多方法，
*
* 1-明确动词的指令集，因为人类的动词不但数量少还有限，又精确。非常方便 第一步索引先。
* 2-单句分解指令集，因为单句是一个完整逻辑句，方便以后各类歧义 句型 复句句型首先转换
* 为单句再执行即可。
* 3-最大的价值在 1 和 2 可以直接元基 索引 IDUQ 分类即可。方便我的花语系统加速。
* 如六元-StudyVerbalMap
*/

```

```

/*--函数编码时产生的问题和问题描述以及思考--

```

```

* （首-先，一，开始，于是，顺其自然，）
* （将，获-取-得，授权，选择，确-定-保，认-准-定，标-记-出，拿-出-到-来，把，）
* （表 表格-单-库，矩阵，文-档-件，对象）
* （进行 执行 跟进 更近 更进 数据 智慧 逻辑 选择 操作 确认）
* work domain out later.
* NE.app_S.workVerbalMap.setHumanTalk(command, NE);
*
* 准备整体将注解提取到函数体外部，避免无效计算，这样做需要进行标识。如关键字+~+处
* setHumanTalkAfterNewBusinessTest处，避免修改频繁，不要写行数。
*
* 举例
* 关键字 getCartesianRelationShipFromHumanTalk 处 思考
* 思考1 - 当构造混合中文数字提取转换匹配后，进行归纳格式化map，这个map则需要在这
* 这一层进行和分词结果整合。这种逻辑属于ETL类型逻辑， command_V._IMV_SQL_SS

```

```

* command_V._IMV_SQL_SS_Q
*
* 思考2 - 所以这种逻辑以后可以更进分解swap成用tinshell OSGI ETL节点来中文节点分层
* , 以后人工智能的基础思考模型就稳定了。然后元基索引 使用频率统计排序归纳, 创造一个
* 自然选择的计算模拟环境。
*
* 这一层逻辑已经并入了 setHumanTalkAfterNewBusinessTest, 于是注释掉。
* 分词的position要统计char位置, 不是word位置, 不然会不准确 later --trif一下
*
*/

```

TVM中笛卡尔关系逻辑编码

14.1.4--FastCartesianIdentifyTest

```

/*--函数编码时产生的问题和问题描述以及思考--
* 思考2点
*
* CN 1- 之前的意识错误纠正, 主要是思维方式的纠正, 之前我把2018年分词还是用老版Lucene
* , 把落后归咎于业界的不争气。在这一点上我还是要诚恳地道歉, 做人应该符合科学的方法论和
* 合理的制度规范 去 思考论题。不能激进。好比我写函数思想一样, 也不能图快和激进, 因为我
* 的目标一直很明确, 瞬间转移和永生, 只是人工智能, 元基编码恰好是这个探索过程的中间
* bonus 产物。
*
* 2- 文字的描述内容目的非常明确, 就是让计算力有浓度。计算力在某种 观测的面上可以用语言
* 的形式来体现。语言是可以提炼的, 所以计算力有浓度。比如笛卡尔关系, 为了更准确的提炼TVM
* extension 花肽语指令。可以将源码的这部份提出来, 专门设计针对各种语言, 输出计算相似
* 的指令片段标注。最终形成自适应的指令片段而不是人为地去定义这个片段。比如目前的8个肽
* class 还是我手写的, 之前的3000多花语class也是我手写的。通过仔细分析, 不难得到很多
* 仅自适应逻辑计算, 就可以分类的属性, 比如相同片段, 相同的词性, 相同的含义, ...。在map
* 归纳下都可以谨谨有条地归纳出来。
*
*
* EN 2- Type of observation, to raise and make an ability of computation more ratios of concentration.
* Assumption a format of An ability of computation could be a detailed human's literacy-language.
* The language has been extracting the purity of cognition every times since was used, the ability of
* computation has been mining well in production's efficiency-domain simultaneously. And recently
* one of the effect way about making a promotion of exporting out the Cartesian's relationship-logic
* with specially test models, to better for continuing sub-designs. Thus models could do well in self-adapted
* and initialed as valuable sections of TVM's derive-construction.
*
* Could start a few sections work below.
*
* package name-- package S_A.SixActionMap; file name-- WorkVerbalMap function name-- findSubject
*
*/
// to do a swap..

```

```

/*--函数编码时产生的问题和问题描述以及思考--

```

```

* 首先提取已有的函数通过 getCartesianRelationShipFromHumanTalk 计算细腻地处理

```

```

* 好 WorkVerbalMap 和他的 SVO 对象，笛卡尔的关系可以通过对象进行PCA，之前的排序就用
* 上了，排序后选择。选择一些有代表性的，没有特殊符号的，明确动词的关系组，然后在这些关系
* 组中通过 getCartesianPromotionTVMFromRelations进行跟进筛选归纳有价值的逻辑。
*/

/*--函数编码时产生的问题和问题描述以及思考--
* 这个函数用于 filter 笛卡尔map 中的 大量无用的成员，每减少一名成员，之后的 计算就增
* 加一分速度和性能。
*
* 都到这一步了，价值就非常明显了。1 首先我找出了一个严重的问题，就是我的笛卡尔关系是拆
* 开的分类的关系，所以我应该设计一个新的全局关系比如功效==搜索 然后再区分关系 功效-搜索
* 和 搜索+功效区分，所以我漏掉了一个全局关系Map，目前还看不出价值，但是处理海量关系一旦
* 有概率这个计算全局逻辑，那么将是巨大的计算浪费。
*
* 2 其次观测发现我的笛卡尔关系变量描述不规范 条件+功效:9:5:_stringNoun1_stringVerb10
* 这个例子如果更加适应计算，后后缀应该是数字代号描述会更好，比如条件+功效:9:5:4:5:1:10
* ,统一格式可以探索一劳永逸的表达方式，加速思考和行为编码。
*
* 3 发现这么多当前可以做的事情，说明计算哲学的价值巨大，一个人的研究能力是他的母语水平，
* 一个人的母语水平来自与他对事物的简短文字描述能力。这个能力来自定义 和 辩证的学习力。
*
* 4 跟进思考，首先剔除和筛选掉 全部是符号的笛卡尔关系成员，再筛选掉含有关键符号的关系
* 成员。再筛选掉缺失一半关系的成员。不是删除，是筛选，所以我还要设计3个map装载这些垃圾。
* 将来有用。目测一半的数据被清理掉，之后性能指数翻倍。
*
* 5。。。
*
* 6 完全正确的源码计算，正确地输出，稍微一个变换思考，一下出现了好多问题，而且这些问题
* 都能有当前环境下的标准答案。。
*
* 很多人不知道自己当前要干什么，并不是能力不行，恰恰是能力太强，譬如我，只是当时不知道
* 自己的兴趣爱好是什么。因为什么事情都思考一下，生活会很累很乏味，只是逃避而已，但是一旦
* 知道了进行有选择地思考，就开挂了。
*
* 辩证思考的魅力 - 罗瑶光
* ----
* 开始筛选出含有特殊符号的map，命名为relationsASCII，从command_V._IMV_SQI_SS
* 中提取 于是开始思考，是先提取再计算还是先计算再提取？
* --先提取再计算，加速主要关系输出。
* --先计算再提取，增加细腻属性关系。
* 于是跟进思考，含有特殊符号的relationsASCII属性对语言分析有什么充份且必要的价值影响？
* 1 代号使用价值，以后有变量模型的语言有代换价值。2 later。。
* 这些符号没有严谨的词性意义，在SVO中的POS作用表达不强，于是采用先提取再计算的逻辑。
* 很多时候在非应激表达的劳动中，说服我做决策的从来不是我的主观爱好，而是客观辩证思维。
* ----
* 关键字 string.contains("*" 处--笔记
* 歧义数字变量关系
* 思考 *号代表输入的文字中有变量，于是分析这个* 和处理后的变量是否需要同时存在与关系中。
* 首先早期没有处理变量的时候，变量字符都是断开的，增加了大量的笛卡尔关系。增加了变量处

```

* 理后，有了*，但是变量的position是char的位置来计算的，不是切词的list位置计算的。
 * 那么position也是规范准确的可用于计算的变量。* 在笛卡尔关系中的意义代表变量与词汇的
 * 关系。如果筛掉*，用原数字来表达，那么还要增加一层关系，数字是不是变量，再思考变量和
 * 词汇的关系。但是通常数字和词汇本就是变量的意思。那么这里可以先筛掉*，加个slash，以后
 * 要参考*的笛卡尔条件，去掉slash。目前我的CreativeVerbalMap 十六元基索引
 * TVM extension 和 blooming Action 都没有*的参照条件，可以注释下先筛掉。另外为了
 * 一个 * 增加一把笛卡尔关系词汇组其实也是不值得提倡的编码逻辑，可以采用含*的boolean
 * 标识，做条件编码，更规范的管理含有变量的逻辑句。于是思路就清晰了 * 属于语法 构造关系
 * 。先并入relationsGrammar关系中

```
*
* WordFrequency wordFrequency = workVerbalMap.command_V._IMV_SQL_SS
* .getW(string);
* workVerbalMap.command_V._IMV_SQL_SS_temp.put(string,
* wordFrequency);
* continue;
*
*/
```

/*--函数编码时产生的问题和问题描述以及思考--

* 提取子功能测试

* ---

* 在这个逻辑思维中，我一直在思考--碎片记忆与段落记忆的方式不同 则对 函数的计算影响也不同。
 * 如何有效地区分和管理这些不同所带来的问题集合。这是一个非常抽象的难以描述的逻辑和概念
 * ，如果不经反复地推敲和细腻的文字描述，很难发现这些点。可这些点偏偏价值巨大。在科学
 * 上，任何名词的定义是没有好似，好像，类似这种词汇思维的，如果有，那便是对基础认知
 * 不足，或者是当前的思维无法分辨某一类现象，又或者是这类现象还没有被严谨地归纳和定义。

*

- * 1 有效地规避函数级别的死循环 和 类级别的死递归。
- * 2 合理地分配函数在堆栈中的序列构造，减少假缓冲和无效逻辑。
- * 3 剔除大量的无关变量和冗余的编码片段，目的性更强。
- * 4 方便更进研究和扩展。

* --

* 准备定义笛卡尔的符号规范

* 1 因为一开始采用 全局的去动名词的全局笛卡尔关系，所以四个

* cartesianWorkActionsRightsVO

* cartesianWorkActionsRightsSV

* cartesianWorkActionsPositionsVO

* cartesianWorkActionsPositionsSV

* 里面的结果是一致的，所以可以剔除减少75%的计算量。

* averagePositionNoun 比较不再有价值需要进行全补充关系

* --

* 剔除并不是删除这些计算量 而是重新利用。原来的+和-是动和名的组合，因为是全局关系，
 * 不再有这个逻辑了，因为人类语法动名词可以变换。动词本就是名词的一种弱属性。
 * 那么+和-可以重新定义为 文中的词汇关系 先后为+，后先为-。这个价值可以用来观测词汇
 * 的因果发生现象。那么这里出现第二个问题便是 当一个指令句出现多个相同的字，就+-无效了
 * 如果做平均数来计算，那么平均数仅仅是代表距离的分布关系，而不是前后关系。

* --

* 在哲学计算的层面上，上面思考的内容属于新的关系更近分析，所以进行归纳，关系分层，
 * 这里仅用+做笛卡尔全局计算先，新的关系和新的条件，用新的决策树函数逻辑来处理。

```

*
* --那第一层决策关系函数处理的逻辑 就没有了- 和动名词之分。
* --VO map全部省略，SVO逻辑区 先 减少75%的内存占用。
* --这种操作先保留，因为动名词的全局笛卡尔逻辑全省略有风险，
* ---
* 主要成分价值map
* cartesianWorkActionsRightsVO
* cartesianWorkActionsRightsSV
* 仅保留PCA，VO map全部省略，SVO逻辑区，减少80%的内存占用。
* 当前main功能测试通过。Tinshell测试通过。华瑞集admin界面测试通过。
* WorkVerbalMap_X_S 195行处PCA过滤，以后用到条件再解开slash。
* ----
* 筛选前主要关系数-->13
* 筛选后主要关系数-->8
* 在含有各类特殊字符的输入中 主要成分计算量减少40% 价值巨大。
* 在纯中文的规整字符输入中 准备应用于tinshell 和 tinshellSeparateTest 测试。
* 结束
* 思考-条件和逻辑的分层筛选，对特殊的问题会产生一些盲区。可以叫bug。
* 稍后在处理逻辑内容时，要考虑这些盲区，写FIX管理关系函数。
* 并不是现在输出正确就是一直准确。环境是变化的，所以逻辑也应该遵循这个
* 变化中的长期稳定类规律去自适应。
*
* */

```

14.1.5--WorkVerbalMap

```

/* 一些逻辑不应该出现在电脑上，只能文字出现在书本上。就因为电脑内置蓝牙wifi声卡接口
*，我就不爽。不管了我就当写书一样就是了。--罗瑶光 trif
* --1 华瑞集的所有函数通过元基花进行注册登记，注册登记的函数进行24组十六元基编码索引分类
* 模拟染色体 可以序列化索引和编码所有函数。AOPM VECS IDUQ 稍后就能用上了。
* --2 函数调用接口通过元基枝进行归纳分类，归纳分类的接口通过功能测试进行功能分类成六元分析机
* 模拟生活领域的生存技能。六元分析机 -爱-学习-帮助-创新-安全-工作-
* --3 通过笛卡尔关系计算将六元功能分析机 与 24组十六元基编码索引分类函数 进行指令集熵化结果。
* 构造一个内存大脑tin map 保存这个熵化状态。
* --4 增加面向对象继承孢子形态 OSGI形态 花语形态 手工编码形态 测试函数融合 4种形态扩展插件
* 指令集和VPCS业务逻辑。
* --5 通过一次新陈代谢逻辑和二次新陈代谢逻辑来缩进函数名，文件名，标题名，变量名，宏名，类名
* 增加计算效率，肽展公式进行优化关系。
*
* --这上述5点的-DNA元基催化与肽计算-过程保证了以后华瑞集函数指令系统进行仿生拟态杂交仅仅融合
* 测试文件和测试文件的统计自然选择确定那些函数和指令在杂交后保留，优先，剔除，等操作逻辑即可。
* 目前这个框架越来越规范和完善。离不开详细的文字描述。这是哲学的分支。慢慢开始意识到，
* 人类的进化过程中一个明显的导向是对时间事物的具体描述能力。这个能力也是认知能力。所以年轻
* 人一定要学好语文，特别是其中的散文 叙述文和议论文。
*
* --写到这我开始思考笛卡尔关系的去重逻辑，笛卡尔关系属于全耦合关系，不符合量子计算的规范。
* 关于笛卡尔关系优化，我想到了很多点，
* 1 首先便是去重，可以构造一个临时map来辨别当前子关系是否被重复计算。定义cartesianLooped

```



```

* 2 局部微分化，将段落句拆分为单句集，子单句拆分为 DNN词汇句。DNN词汇句做分词笛卡尔关系计算。
* 3 多层笛卡尔关系做sigma熵化，关系的过程变量进行微积分优化求导，结果作加法累积。
* 4 稳定功能测试后进行 -DNA元基催化与肽计算- 上述5点算法融合。
* 5 AOPM VECS IDUQ 指令集索引 稍后就能用上了。
*
* --于是跟进这个5点小目标。
* 关于DNN的词汇归纳思考- 仅仅在子单句的长度超过30字-精度30可设成变量可自由设计-的条件下
* 触发DNN替代原句分析。避免繁琐计算，达到有效计算加速的目的。
*
* */

/*
* 发现很多变量在逻辑意识优化后，关系也变化了，一些计算产物之后都没有用到了
* 于是先保留，方便之后的逻辑更近。
* */

/*--函数编码时产生的问题和问题描述以及思考--
* 新的商业测试接口既然写了那就要用。作为世界记录的7年保持者，我罗瑶光要做的
* 只有2件事情，1挑战我自己和期待对手挑战我，2 改变我自己对以往事物的评价，
* 给自己传道授业解惑。什么时候被业界打倒了，我就退休。好多东西等我玩。写代码
* 只是我的兴趣爱好。别抽象我。我只是个凡人。
*
* 关键字 getWordFrequencyMap 处 --思考
* -1 词频 归纳
* -2 词性 归纳
* 之前逻辑是 所有词性词汇 搜索 归纳
* -未知 词汇 入 NE.app_S.workVerbalMap.unknown_map.put(string,
* true);
* 其他 入 mapSearchWithoutSort.put(string, wordFrequency);
* 测试逻辑是 名 动 形 副 修正归纳 map-string-wordFrequency
* 缺少逻辑是
* 增加其他词性 map 同时统一入 mapSearchWithoutSort 即可逻辑有很多种，
* 我选择 都做一遍，然后loop 替换即可我的动机是确保包含所有形式的 完整计算关系。
*
* --函数编码时产生的问题和问题描述以及思考--
* 关键字 DemoAfterPOSTest demoAfterPOSTest = new DemoAfterPOSTest(); 处--思考
* 思考，当一个原来的词汇关系系统计算中，纠正了副词的准确性，那么原来的函数中形容词的词数
* 就会大幅地减少，如果之后的跟进计算用到了形容词，而没有用到副词，那么条件的精度会增加，
* 而过滤数也会增加。这样的计算强调语法包含，质量提高，但性能降低。提高性能的方式是增加
* 副词逻辑的计算函数集。--罗瑶光
*
*
* DemoAfterPOSTest demoAfterPOSTest = new DemoAfterPOSTest();
* List<String> setsInput = demoAfterPOSTest.testPOS(command_V._IMV_SQL_SS_, pos);
* List<String> setsAfterInput = demoAfterPOSTest.testAfterPOS(setsInput, pos);
*
* --在测试文档中已经设计了更为精准的 DemoAfterPOSTest 服务，于是这里进行替换下函数，
* 看效果如何。testAfterPOS 的逻辑是 出现单字的 同词性相联的进行合并，但是这个格式
* 在当前的tinshell逻辑中不能采纳，于是需要稍微的变形。编程可采纳的格式。
*

```

```

* --问题1 demoPOSTest.testPOS 已经将改变的词性修正 到wordFrequency 类中还注册了
* char position 直接变形比较麻烦，于是开始梳理思路。
* --解决方法 首先分解 demoPOSTest.testPOS 为两个逻辑，先修正pos 变成 含有/的 list
* --再list组合修正。--最终将修正的list 处理成 wordFrequency 格式list。
*
* --关键字 if (false == command_V.initedArabicNumber) { 处 --笔记
* 在进行分词前进行数字提取过滤，得到数字类nums和序次的map然后过滤掉这些数字
* 的string进行下一步的操作，如果有alfs的提取任务，就alfs也用这个逻辑处理。
* --罗瑶光
*
* */

```

```

/*--函数编码时产生的问题和问题描述以及思考--
*-- 阿拉伯数字加进map中，归纳在子函数这，是方便函数被第三方接口封装调用方便。
*
* S_logger.Log.logger.info("'" + "--" + command_V._IMV_SQI_SS_Q.size());
* S_logger.Log.logger.info("'" + "--" + command_V._IMV_SQI_SS.size());
* Iterator<String> iteratorNumbers = command_V._IMV_SQI_SS_Q
*     .keySet().iterator();
* while (iteratorNumbers.hasNext()) {
*     String temp = iteratorNumbers.next();
*     WordFrequency wordFrequency = command_V._IMV_SQI_SS_Q
*         .getW(temp);
*     command_V._IMV_SQI_SS.put(temp, wordFrequency);
* }
* S_logger.Log.logger.info("'" + "++" + command_V._IMV_SQI_SS.size());
*
* 阿拉伯数字处理--笔记
* 1 精确词汇pos函数
* 2 精确词汇笛卡尔 取缔之前的老快速 map 频率
* 3 精确词汇rnn 和 position loop unknown
* 4 精确词汇的mapping肽指令集
* 5 局部替换即可，价值可识别12345和英文abcde 方便人类语言中入参识别。
*
* */

```

14.1.6--WorkVerbalMap_X

```

/*--函数编码时产生的问题和问题描述以及思考--
* 在计算哲学和语文意识中，非动词类词性组合可以算法省略，因为不构成指令集核心成分。我先不管，
* 在计算意识中，单字的价值不打，但是在歧义处理中，单字是普遍存在的，如果下面函数执行，那么
* 需要有精确的语法做铺垫，如果没有，那么就要进行概率累积评估，累积打分算法逻辑来灵活驱动。
* ----
* 思考--关于 6到-9 进行行数筛选指令句 的条件分析，这里6到的组合是因为 6是一个char，到也是，
* 小于2就并在一起了，成了6到。于是开始更进思考，这里 如果是600，那么出现了一个问题，便
* 是组字的拆卸问题，这个逻辑进行捋一捋，可以的得到一个流程，便是 数字+一个字词变成高级
* 笛卡尔的渡的关系。而不是之前的 rootM.length() < 3. 这样做需要一个精度来进行控制。会让匹配
* 更灵活，于是开始修改。因为词汇分词后是最大也就4个字。但是一旦包含数字就不同了。所以
* 可以省略。

```

*/

/*--函数编码时产生的问题和问题描述以及思考--

* 思考，这里的 + - 符号 + 是position，-是connetion，在词性组合中 +是动名，-是
 * 名动，形成原因是我不断地优化引擎，用最简添加法方便迅捷编码，为了区别混淆，于是又 +
 * 上改为++，-改为--来区分。那么在函数map中也应该进行区分，于是优化下最终结构 一个DP
 * 代表双字+—POS组合后的+-RNN组合。输出格式为 DP(+-) DP(++ --) DP(+-) 复杂笛
 * 卡尔完整关系模型，那么在normalizationalWorkActionsRights.put的函数中就可以在
 * 排序后进行精度筛选即可以避免过多的内存资源堆栈浪费，之后指令集按照累积打分来驱动计算
 * ，精度筛选所造成的误差缺陷也能 得到最大限度的弥补 -罗瑶光
 */

/*--函数编码时产生的问题和问题描述以及思考--

* 关于复杂RNN关系 我引擎这里会逐渐用不到，因为DNN的重心分析加上语言的单句分解，是我
 * 的花语计算逻辑趋势，以后复杂的句子首先走短句分解而不是走全局关系，那么这个复杂关系
 * 价值将体现在修正和补充环境里面，之后根据算法优化的BPM结构不断校正补充函数 -罗瑶光
 */

14.1.7--WorkVerbalMap_X_S

/*--函数编码时产生的问题和问题描述以及思考--

* 开始设计高级笛卡尔关系 分
 * 稳定后在思考变量的归纳方式。
 * 关于精细化map的价值，举例50个子集成员进行处理，有20个单子集合并成20个双子集，那么当时
 * 最早的逻辑是 替换组合子集 = $(50 - 20 + 20) * (50 - 20 + 20) = 2500$ ，这种方法在处理从- 到-
 * 问题时候失效了，于是后修改为 笛卡尔关系 包含本身 累积是 $70 * 70 = 4900$ 。这种方法ok但速度翻了
 * 1倍，于是现在准备采用高级map，思考--当 map 分区后，原系是 $(50-20) * (50-20) = 900$ ，
 * 单子集 $20 * 20 = 400$ ，合并双子集 $20 * 20 = 400$ ，原系-单子集 = $30 * 20 = 600$ ，
 * 原系-合并双子集 = $30 * 20 = 600$ ；单子集-合并双子集 = $20 * 20 = 400$
 * 共计 $900 + 400 + 400 + 600 + 600 + 400 = 3300$ ，效果就出来，不但精确了双子集的价值
 * 因为包含 原系-单子集 逻辑，所以还依旧保留了从- 到-的问题解决方案。
 *
 * --稍后编码从理论走向真实论证，注意规避一些 PCA 逻辑误区。估计有200行源码要增删构造。
 *
 * --哲学问题论证--任何理论充分的奇想和冥想在没有一个真实具体的客观参照事务作为论据对象，
 * 都属于某种形式的空想。不能否定其价值，所以属于一类可以被尊重的无效计算。
 *
 */

/* --函数编码时产生的问题和问题描述以及思考--

* 为什么现在不设计成implements接口，因为目前没有明确六元函数定义域规范，
 * 以后批量计算模型会CE按XCDX分解来做计算加速。
 *
 * 之前做了商业测试文件，里面有优化校准副词的函数逻辑，那么既然测试写了就要用到，
 * 于是我的思维是那就直接用阿，于是就把测试函数 new出来然后将Noun和VERB取代
 * 这里的nounInText和verbInText，准确率就提高了很多，并不代表结果会更精确，
 * 所以替代后要将所有流程都校准一遍。 --罗瑶光
 *
 */

```

/* root_v += stringVerb;
* root_v += "-";
* root_v += stringNoun;
/* --函数编码时产生的问题和问题描述以及思考--
* CN
* --十六元基索引逻辑
* 1 名词索引价值用于古拉丁英文索引元基编码加速特征片段识别。
* 2 动词索引价值用于TVM指令集快速降维面化 进行 触发计算识别。
*
* EN
* 16 Initon indexing
* 1 Latin noun's verbal exchange could make cluster with short , less and same sections of DNA initons.
* better for observation.
*
* 2 Latin verb's verbal exchange could make trigger with short, less and same sections of DNA initons.
* better for computation.
*
*
* 稍后开始整理关系map，分出精细的属性成员出来。
**/

```

14.2 分词前歧义中文混合数字识别模块和

--主要应用在tinshell的TVM命令句中数字识别和分词中切词前的数字识别。

--识别逻辑

14.2.1--StudyVerbalMap

```

/* --函数编码时产生的问题和问题描述以及思考--
* --这个函数用于将command string中的数字和汉字进行提取出来，形成一个map，如果是汉字就进行
* 阿拉伯数字变换。提取map后将原string的对应字符全部过滤掉生成commandWithNumFilters。
*
* --函数中的条件归纳优化。
* ((A & B) | (C | D ...)) = res 见该函数尾巴注释部分。
* --优化 1 卡诺图化简
* 略
*
* --优化 2 迪摩根化简
* !res =  (!((A & B) | (C | D ...))) --
* = (!!!((A & B) | (C | D ...)))
* = (!(! (A & B) & (! (C | D ...))))
* = (!(! (!A | !B) & (! (C | D ...))))
* = (!(! (!A | !B) | !(! (C | D ...))))
* = (!(! (!A | !B) | (C | D ...)))
*
* --优化 2 离散化简
* Map<String, Boolean> chineseNumberSets = new HashMap<>();
* chineseNumberSets.put('零', true);
* 稍后提取出去成为stable map
*
* --println函数走图形打印机，并发工程记得注释掉或者用其他的classic观测API--如log4j
* --函数中的更思考，这里应该直接处理掉数字变换，就能很好地避免-年-月-单字成为词汇的问题。
*

```

```

**/
//我稍后会将所有计算区的main函数整体移到测试文件夹下去。

/* --函数编码时产生的问题和问题描述以及思考--
* 今天测试覆盖率思维发现一个问题，问题的描述是我在做16位大数循环测试时候，当测试的循环
* loop到 205602 的时候显示 二十五万零六百零二，于是我单个测试，发现上行函数是
* 二十万五千六百零二，下行一个function call 进去就成了 二十五万零六百零二，百思不得其解。
* 我开始--思考
* 1 -- 是内存不足 边界堆栈buffer裁剪 导致寄存器冗余归纳出错？
* 2 -- 我眼花？于是重新测试 loop 竟然还出现了 -零零- 连续。
* 3 -- string buffer在快速计算时候log和write println等一样需要flush一下？
* 奇怪jdk文档都没有这一条啊。
* 4 -- 电脑没有删除out 还要 clean？
*
* 解决方法
* 1 带着这个问题，我能做的就是将华瑞集工程中 复杂string计算模块 逐步变成string builder模块
* 2 string builder模块中 常见乱码的模块部分 逐步变成 byte array模块。
*
**/

/* --函数编码时产生的问题和问题描述以及思考--
* symbolsSwapNumericsByLength 与string的 长度一致 增加分词的char position
* 准确度，因果增加笛卡尔关系的精确度。
*
* --关于混合中文歧义数字描述
* 因为有些用户喜欢写100万2000这种标识，就不用一百万两千和1002000这类规范的。
* 所以我在这个if里面之后还要设计个阿拉伯数字转汉字的数字翻译机。逻辑是先拆分数汉，
* 再翻译数变汉，最后组合全汉输出即可。
* --loop 找出阿拉伯数字，然后翻译拼接
*/

/* --函数编码时产生的问题和问题描述以及思考--
* 病句歧义分析123万200亿000 也可以理解为一百二十三万零二百亿仟，但是这里出了一个问
* 题，如果后面接123，
* 如123万200亿123，那么这里有两个意思，1是一百二十三万零二百亿 个 123，
* 要做乘积，乘123，在这种计算逻辑环境中，数字提取的将不是稳定的数字提取逻辑，而是数字乘法
* 函数的提取，提取的是一个要计算乘法的逻辑。可以理解为 病句歧义分析含有算法逻辑的数字函数。
* 所以不考虑，按完整数字逻辑来获取。按拼接法 123万200亿000 需要改为 123万200亿，再举
* 个例子123万200亿2000则保留末尾2000，因为2字有数字拼接逻辑意义。--罗瑶光
*
*/

```

14.2.2--StudyVerbalMap_Q

```

/* --函数编码时产生的问题和问题描述以及思考--
* --S_logger.Log.logger.info("'" + "混合数字字符预处理锁定-->" + string);
* 因为有些用户喜欢写100万2000这种标识，就不用一百万两千和1002000这类规范的。
* 所以我在这个if里面之后还要设计个阿拉伯数字转汉字的数字翻译机。逻辑是先拆分数汉，
* 再翻译数变汉，最后组合全汉输出即可。--罗瑶光
*

```

```

* --首先构造一个getChineseFromNumerics，我设置最大数为16位，因为之前的
* getNumericsFromChinese也是16位，我电脑最大也就8位。大数运算我不cover。
*
* --首先开始思考，假定这个number是一个标准的整数，因为小数逻辑简单，
* 只要直接翻译char即可。负数只要前面价格符号即可。
*
* --于是开始分解，4位的千万位一个区间，那么是4个区间，符合普通人理解。
* numberParser[0]=0~9999
* numberParser[1]=0 0000~9999 0000
* numberParser[2]=0 0000 0000~9999 9999 9999
* numberParser[3]=0 0000 0000 0000~9999 9999 9999 9999
* 这就简单了，直接加个戳组合即可。
* --论证了 计算哲学 适合 所有定义类逻辑的描述。
*/

```

14.2.3--StudyVerbalMap_X

```

/*--函数编码时产生的问题和问题描述以及思考--
* 先归纳汉语 归纳方式 DNA十六元基语义解码规范。 AOPM VECS IDUQ TXHF
*
* 人类的动词单字不多，但是词组很多，于是更近思考，是否有必要进行单字化。
* 否定论的思维，我就否定单字的价值，让多字词用contains条件来熵化单字计算结果。
* 增加准确度。十六元基编码在索引动词的范式上面，比较全面，这样S元基就被释放出来缓存
* 未知和名词，S作为吸收机制，X来驱动关系，其他元基作为消化机制，T作为驱动逻辑。
* 这些关系和逻辑在千百年的人类进化中动词相对比较稳定。方便我下一步编码。
* 比如标记颜色 的花语驱动就可以增加 --O+颜色-- 这个指令，之后古拉丁文编码，颜色，
* 彩色在元基编码后都有V和Q元基相似片段，就可以减少指令集的扩充逻辑。大幅缩进花语
* 元基编码函数集。
*
**/

/*--函数编码时产生的问题和问题描述以及思考--
* 关于SMV的使用方式，首先我的主要动机是用于做缓存使用。因为NE不够灵动。我想把NE作为
* 数据库使用，每次结束工作，会txt文件记录NE，系统工作的过程是NE在map中的计算，读取
* NE的txt文件。这个txt文件可以方便我每次的优化变量中的数据，了解NE变量集合在计算中
* 的波动过程，为下一步做铺垫。因为用java，我也产生了很多疑惑，首先是我不能清楚地得到
* 变量出现的地方所对应的函数所在的具体的包名。getPackage是当前的class对应的函数所
* 在的具体的包名。缺了一个级，所以我在思考当年我在印度学C语言的时候为什么会买本java
* 来看，因为2008年不碰java，我就会一直搞C和汇编。这十年的出现的问题等于0. 所以当初
* 是什么动机让我买java，因为我印度基督大学当时没有java课程。
**/

```

14.3 时函数应用模块

--主要用在频率滤波上

14.3.1--LYGAFDCTDFFT_FTest

```

/*--函数编码时产生的问题和问题描述以及思考--
* 这是一个 时微分 时叠积 函数 在 主流滤波中的 观测和优化 内积噪 方式 真实应用方式，
* 测试main函数demo的test版本，在导入了api之后进行系统集成，然后用下面的对应的函数
* 中源码逻辑进行复制粘贴到工程中，直接运行，即可出结果，源码的逻辑按照输入准备计算的

```


* 参数，然后执行，然后获取输出需要的结果，可以用断点来查看数据，也可以用println来显示输出，方便集成，对程序员友好。系统需要jdk1.8 以上的java环境，本人会把测试的输入输出都注释在这个文件里。及其傻瓜化的流程，方便商业化落地。--罗瑶光
*/

14.3.2--LYGAFDCTDFFT_F

15 申明

--见测试文档中开头首页描述 和 末尾refer部分

16 附属说明 第三方插件包关系依赖，注意兼容版本号问题，

很多内容是拼接而成，之后会承上启下的通畅化其描述语法。

/Trif_DNA-master/mylib/commons-collections4-4.4.jar
/Trif_DNA-master/mylib/commons-compress-1.21.jar
/Trif_DNA-master/mylib/commons-io-2.11.0.jar
/Trif_DNA-master/mylib/commons-math-2.1.jar
/Trif_DNA-master/mylib/commons-math3-3.0.jar
/Trif_DNA-master/mylib/customizer.jar
/Trif_DNA-master/mylib/en_us.jar
/Trif_DNA-master/mylib/freetts.jar
/Trif_DNA-master/mylib/freetts-jsapi10.jar
/Trif_DNA-master/mylib/gluegen-2.4.0-rc-20230201.jar
/Trif_DNA-master/mylib/gluegen-rt-2.4.0-rc-20230201.jar
/Trif_DNA-master/mylib/gluegen-rt-2.4.0-rc-20230201-natives-macosx-universal.jar
/Trif_DNA-master/mylib/gluegen-rt-main-2.4.0-rc-20230201.jar
/Trif_DNA-master/mylib/gson-2.2.4.jar
/Trif_DNA-master/mylib/icam.jar
/Trif_DNA-master/mylib/iText-2.1.5.jar
/Trif_DNA-master/mylib/javabuilder.jar
/Trif_DNA-master/mylib/javacpp-1.5.8.jar
/Trif_DNA-master/mylib/javacpp-1.5.8-macosx-arm64.jar
/Trif_DNA-master/mylib/javacpp-platform-1.5.8.jar
/Trif_DNA-master/mylib/javacv-1.5.8.jar
/Trif_DNA-master/mylib/javacv-platform-1.5.8.jar
/Trif_DNA-master/mylib/jcommon-1.0.16.jar
/Trif_DNA-master/mylib/jmock-cglib-1.2.0.jar
/Trif_DNA-master/mylib/jogl.obj.rc8.jar
/Trif_DNA-master/mylib/jogl-all-2.4.0-rc-20230201.jar
/Trif_DNA-master/mylib/jogl-all-2.4.0-rc-20230201-natives-macosx-universal.jar
/Trif_DNA-master/mylib/jogl-all-main-2.4.0-rc-20230201.jar
/Trif_DNA-master/mylib/json-20160810.jar
/Trif_DNA-master/mylib/junit.jar
/Trif_DNA-master/mylib/junit-jupiter-5.8.1.jar
/Trif_DNA-master/mylib/junit-jupiter-5.8.1-javadoc.jar
/Trif_DNA-master/mylib/junit-jupiter-5.8.1-sources.jar
/Trif_DNA-master/mylib/junit-jupiter-api-5.8.1.jar
/Trif_DNA-master/mylib/junit-jupiter-api-5.8.1-javadoc.jar
/Trif_DNA-master/mylib/junit-jupiter-api-5.8.1-sources.jar
/Trif_DNA-master/mylib/junit-jupiter-engine-5.8.1.jar
/Trif_DNA-master/mylib/junit-jupiter-engine-5.8.1-javadoc.jar

/Trif_DNA-master/mylib/junit-jupiter-engine-5.8.1-sources.jar
/Trif_DNA-master/mylib/junit-jupiter-params-5.8.1.jar
/Trif_DNA-master/mylib/junit-jupiter-params-5.8.1-javadoc.jar
/Trif_DNA-master/mylib/junit-jupiter-params-5.8.1-sources.jar
/Trif_DNA-master/mylib/junit-platform-commons-1.8.1.jar
/Trif_DNA-master/mylib/junit-platform-commons-1.8.1-javadoc.jar
/Trif_DNA-master/mylib/junit-platform-commons-1.8.1-sources.jar
/Trif_DNA-master/mylib/junit-platform-engine-1.8.1.jar
/Trif_DNA-master/mylib/junit-platform-engine-1.8.1-javadoc.jar
/Trif_DNA-master/mylib/junit-platform-engine-1.8.1-sources.jar
/Trif_DNA-master/mylib/log4j-api-2.18.0.jar
/Trif_DNA-master/mylib/log4j-core-2.18.0.jar
/Trif_DNA-master/mylib/mbrola.jar
/Trif_DNA-master/mylib/mediaplayer_2.jar
/Trif_DNA-master/mylib/mediaplayer.jar
/Trif_DNA-master/mylib/mockito-all-1.10.19.jar
/Trif_DNA-master/mylib/mockito-core-5.12.0.jar
/Trif_DNA-master/mylib/multiplayer_2.jar
/Trif_DNA-master/mylib/multiplayer.jar
/Trif_DNA-master/mylib/openblas-0.3.21-1.5.8.jar
/Trif_DNA-master/mylib/openblas-0.3.21-1.5.8-macosx-arm64.jar
/Trif_DNA-master/mylib/openblas-platform-0.3.21-1.5.8.jar
/Trif_DNA-master/mylib/opencv-4.6.0-1.5.8.jar
/Trif_DNA-master/mylib/opencv-4.6.0-1.5.8-macosx-arm64.jar
/Trif_DNA-master/mylib/opencv-android-arm.jar
/Trif_DNA-master/mylib/opencv-android-x86.jar
/Trif_DNA-master/mylib/opencv-linux-armhf.jar
/Trif_DNA-master/mylib/opencv-linux-ppc64le.jar
/Trif_DNA-master/mylib/opencv-linux-x86_64.jar
/Trif_DNA-master/mylib/opencv-linux-x86.jar
/Trif_DNA-master/mylib/opencv-macosx-x86_64.jar
/Trif_DNA-master/mylib/opencv-platform-4.6.0-1.5.8.jar
/Trif_DNA-master/mylib/opencv-windows-x86_64.jar
/Trif_DNA-master/mylib/opencv-windows-x86.jar
/Trif_DNA-master/mylib/opentest4j-1.2.0.jar
/Trif_DNA-master/mylib/opentest4j-1.2.0-javadoc.jar
/Trif_DNA-master/mylib/opentest4j-1.2.0-sources.jar
/Trif_DNA-master/mylib/poi-5.2.3.jar
/Trif_DNA-master/mylib/poi-ooxml-5.2.3.jar
/Trif_DNA-master/mylib/poi-ooxml-full-5.2.3.jar
/Trif_DNA-master/mylib/poi-ooxml-schemas-4.1.2.jar
/Trif_DNA-master/mylib/videoinput.jar
/Trif_DNA-master/mylib/videoinput-0.200-1.3.jar
/Trif_DNA-master/mylib/videoinput-platform.jar
/Trif_DNA-master/mylib/videoinput-platform-0.200-1.3.jar
/Trif_DNA-master/mylib/videoinput-windows-x86_64.jar
/Trif_DNA-master/mylib/videoinput-windows-x86.jar
/Trif_DNA-master/mylib/xmlbeans-5.1.1.jar

个人著作权人, 第一作者: 罗瑶光, 1985年5月25日出生, 性别男, 湖南省浏阳市人.

--浏阳德塔软件开发有限公司-创始人, 法人, 总经理-- 永久非盈利--

yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

(lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)

15116110525- (唯一标识 2018年2月开始使用此手机卡至今从未拓机, 欠费和关机, 手机是实名身份证发票机, 警惕我家附近类似GSM电子狗频率充斥欺诈)

430181198505250014, G24402609, EB0581342

204925063, 389418686, F2406501, 0626136

当前居住地址: 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段 208号阳光家园别墅小区 第十栋别墅 第三层

--最近思考家的地址从2009年的北岭美林花苑9号 改成石霜路阳光花园9号, 又石霜路阳光花园10号, 又大塘路阳光家园10号, 再大塘路一段阳光家园第10栋到现在集里大塘冲路一段208阳光家园别墅小区第10栋, 我在想, 频繁换名, 有问题, 于是准确介绍下家地址。

--湖南省浏阳市集里街道-大塘冲路一段208阳光家园别墅小区第10栋--, 位于大塘冲路一段208阳光家园小区内的别墅房区入口内左侧, 靠近美林花苑入口大门 (美林花苑小区区内见到阳光家园的最前面金属围栏后便是 (10号-面朝建筑左户-罗瑶光家)和 (11号-面朝建筑右户) 2户联体别墅), 美林花苑小区位于大塘冲路一段的十字路口, 十字路线的以点轴心斜对面是--浏阳市大塘实验(普特融合)小学建筑--。

--身份证上地址2011年后姑姑罗庆华一家入住。中国身份证件标注地址是户籍地址不是居民地址, 罗瑶光先生 2018年后一直居住在阳光家园家的第三层。